



AZ-204: Azure Developer Associate

Day 3 — Serverless & Event-Driven Architecture

Azure Functions, Durable Functions, Triggers, Bindings, and Hosting Plans

DAY 3 OF 5

AZ-204 TRAINING

Day 3 Agenda

Today's session builds your serverless foundation — from core concepts to real-world event-driven patterns.

01

Serverless Concepts

What serverless means and why it matters

02

Event-Driven Architecture

Events, producers, consumers, and patterns

03

Azure Functions Deep Dive

Triggers, bindings, and hosting plans

04

Durable Functions

Orchestration, activity functions, and workflows

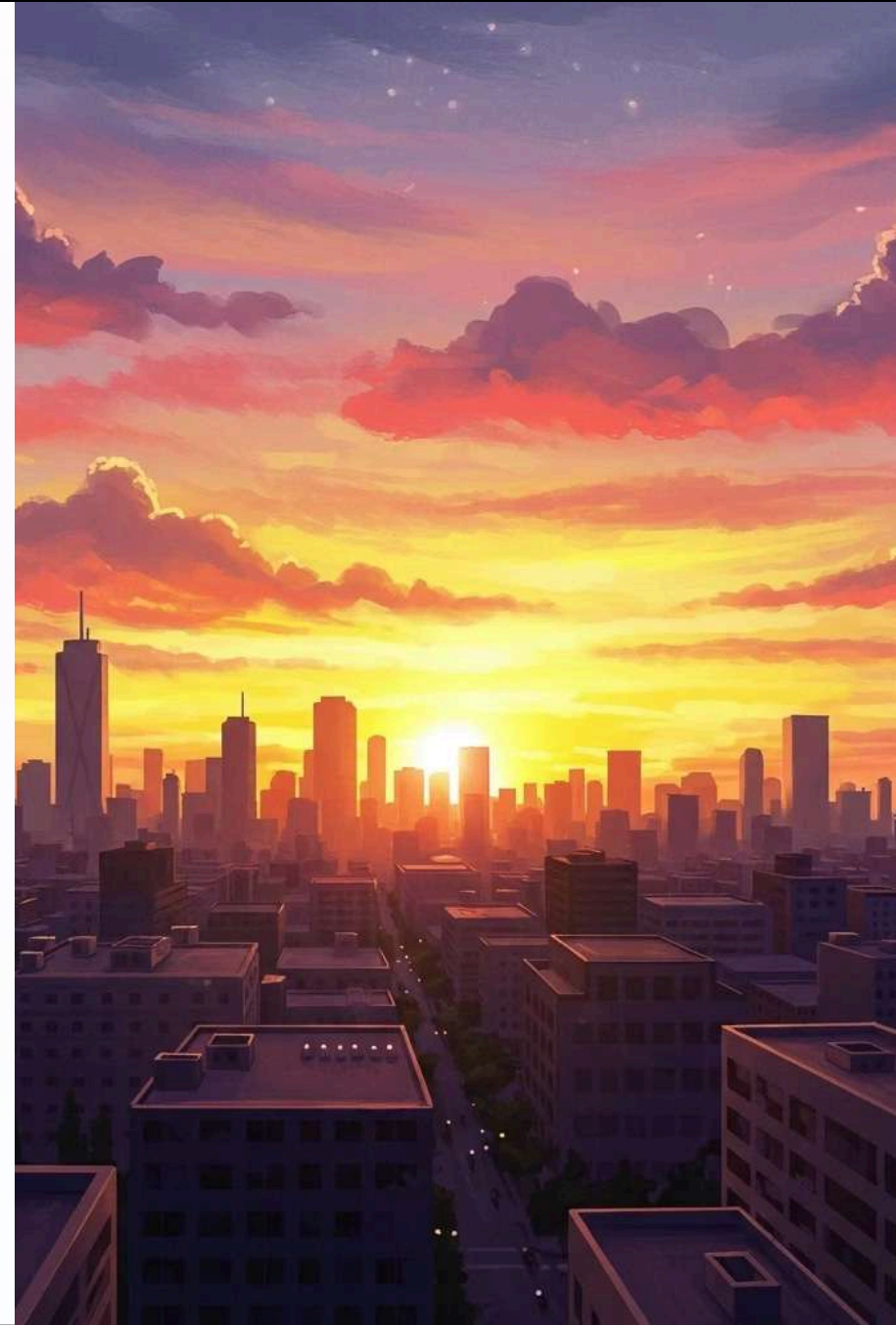
05

Hands-On Labs

Four guided exercises

CHAPTER 1

Serverless Computing Concepts



What Is Serverless Computing?

Serverless does not mean "no servers" — it means **you don't manage them**. The cloud provider handles infrastructure, scaling, and availability automatically.

No Infra Management

No VMs, no OS patching, no capacity planning

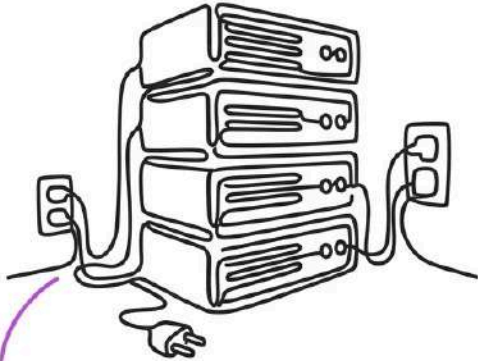
Pay-Per-Execution

Billed only when your code runs — not idle time

Auto-Scale

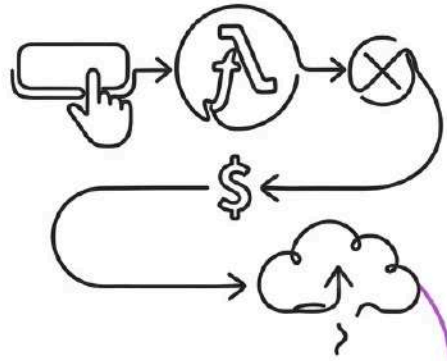
Scales from zero to thousands of instances automatically

Serverless vs. Traditional Compute



TRADITIONAL

Always-on servers
Manual scaling
Pay for uptime
Manage OS and runtime



SERVERLESS

Event-triggered
Auto-scaling
Pay per execution
Zero infrastructure management

For developers, the key shift is moving from **thinking about machines** to **thinking about functions and events**.

- Traditional apps run continuously and scale manually
- Serverless apps run only when triggered by an event
- Cold start latency is the main tradeoff to understand



Azure Functions is Microsoft's serverless compute platform, and the primary focus of today's session.

Key Serverless Benefits for Developers



Faster Time to Value

Write business logic without boilerplate infrastructure code.
Deploy in minutes, not hours.



Cost Efficiency

No idle costs. The Consumption plan includes 1M free executions per month.



Elastic Scalability

From zero to global scale without tuning load balancers or VM sizes. Azure handles it.



Native Integrations

Built-in bindings connect to Storage, Service Bus, Cosmos DB, Event Grid, and more.

An abstract graphic on the left side of the slide, featuring a complex network of glowing blue and purple lines and nodes, resembling a digital or neural network, set against a dark background.

CHAPTER 2

Event-Driven Architecture

What Is Event-Driven Architecture?

In event-driven systems, components communicate by **producing and consuming events** — decoupling producers from consumers entirely.



This pattern enables loosely coupled, independently scalable services — the backbone of modern cloud-native applications.

Core Event-Driven Concepts

Event

A lightweight notification that something happened — a file uploaded, an order placed, a timer fired.

Producer

The component that detects the change and emits the event. It does not know who will consume it.

Consumer

The component that reacts to the event. Decoupled from the producer — runs independently.

Broker

The channel between producer and consumer. Examples: Azure Queue Storage, Service Bus, Event Grid.

Real-World Event-Driven Scenarios



Async Order Handling

Customer places an order → event queued → function processes payment, inventory, and notification independently — no blocking.



File Processing Pipeline

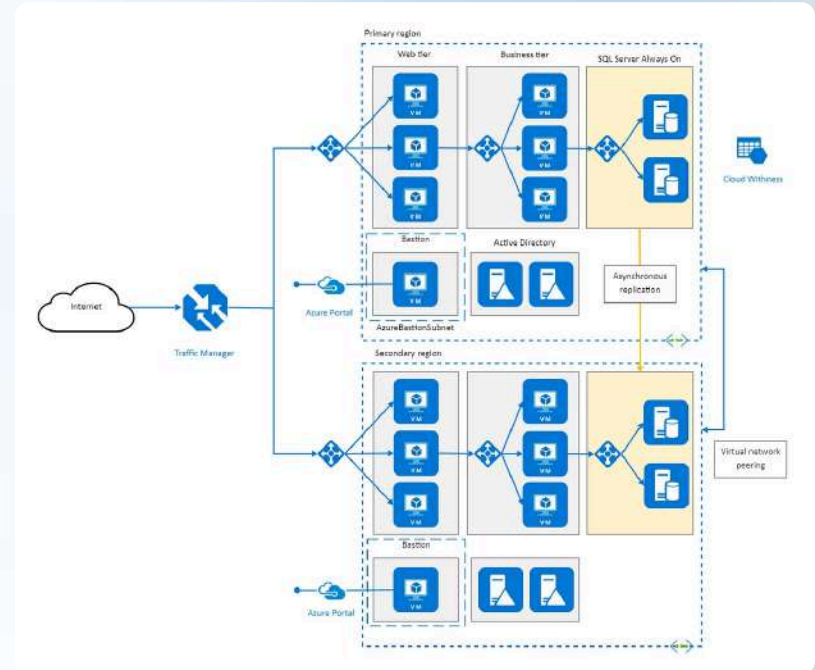
Image uploaded to Blob Storage → Blob trigger fires → function resizes, analyzes, or extracts metadata automatically.

Workflow Orchestration

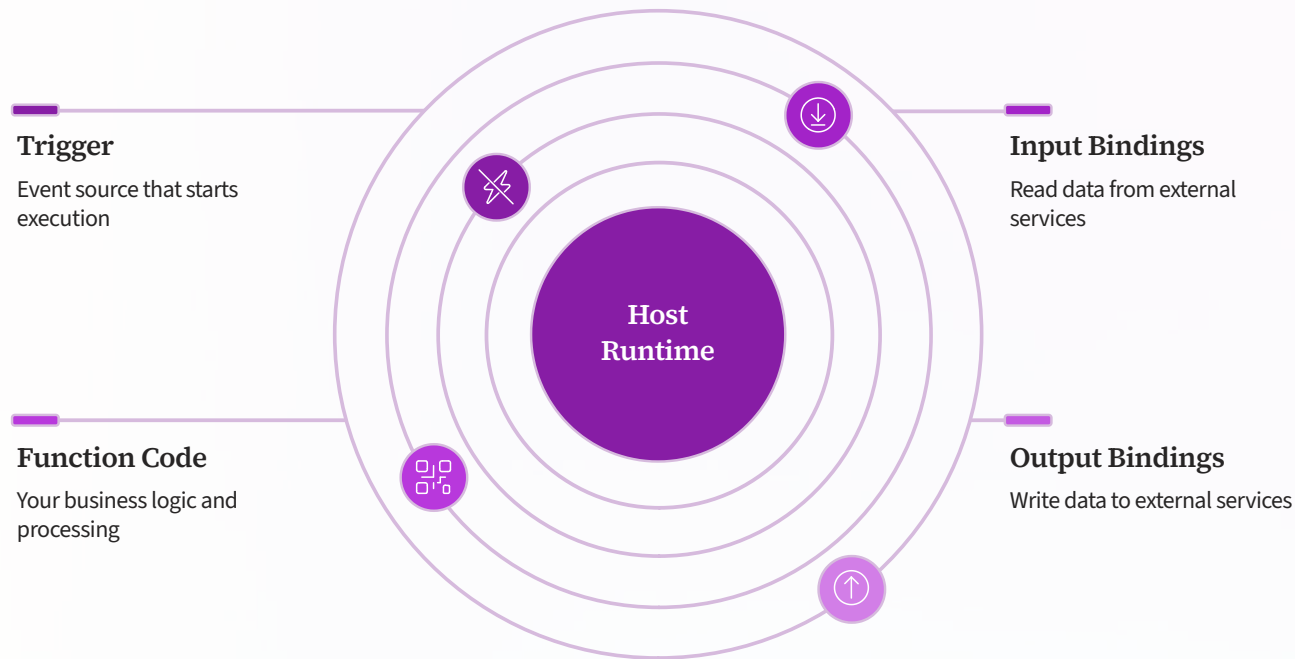
Multi-step approval or data pipeline coordinated by Durable Functions — each step triggered by the previous one's completion.

CHAPTER 3

Azure Functions Architecture



Azure Functions: Core Architecture



How It All Fits Together

Every Azure Function has three architectural layers:

- **Trigger** — what causes the function to run
- **Bindings** — declarative connections to data sources/sinks
- **Function Code** — your business logic, language of choice

The Azure Functions **host runtime** manages execution, retries, and scaling — you only write the function body.

Supported Languages

Azure Functions supports multiple languages. Choose what fits your team's stack — each runs on the same underlying host.

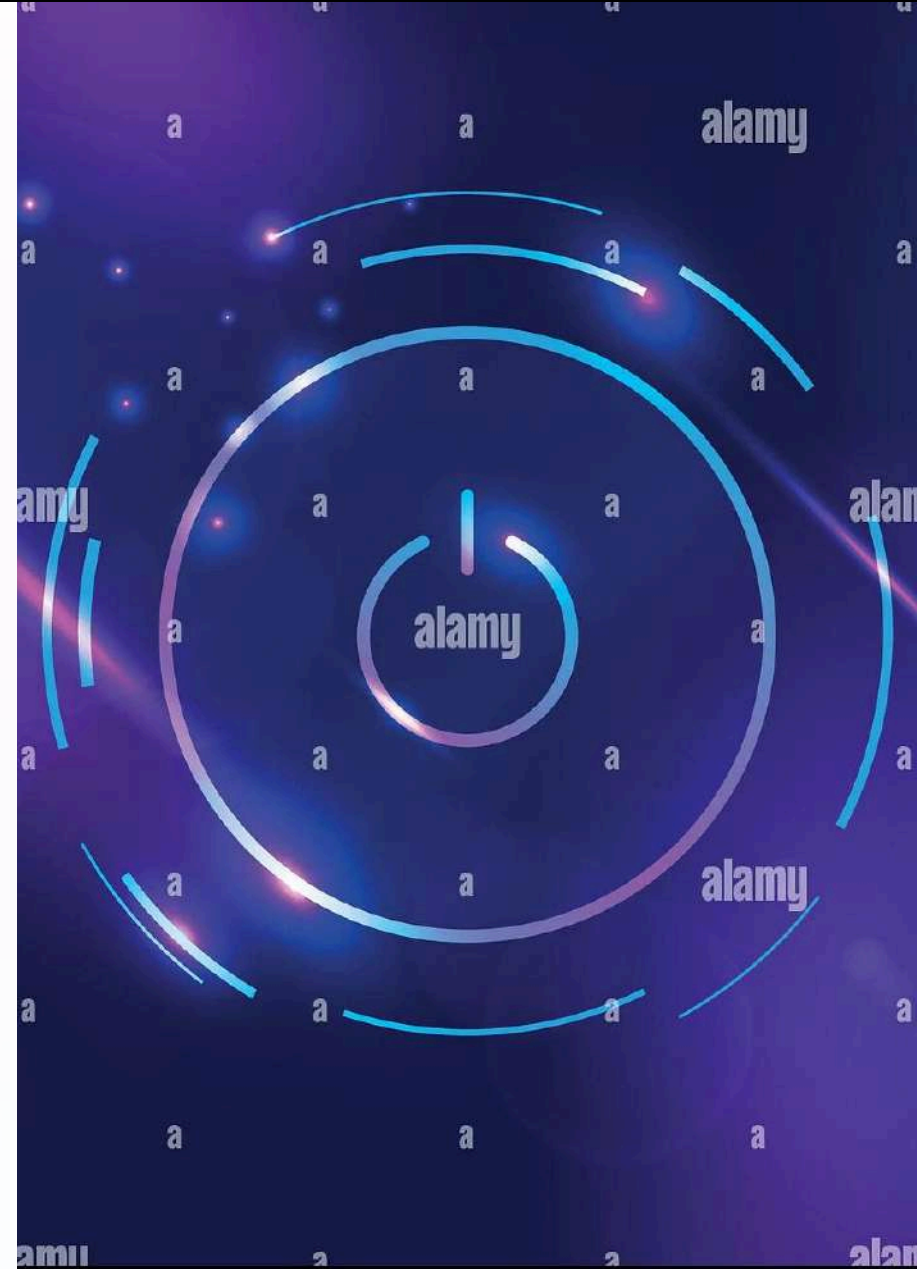
C# In-process or isolated worker model. First-class support.	JavaScript / TypeScript Node.js runtime. Popular for lightweight APIs and webhooks.
Python Great for data processing, ML integration, and scripting tasks.	Java / PowerShell / Go Also supported. Java ideal for enterprise teams already using Spring.



The AZ-204 exam is language-agnostic. Concepts apply across all supported runtimes.

CHAPTER 4

Triggers



Understanding Triggers

A **trigger** defines what event causes your function to execute. Every function must have **exactly one trigger**.



HTTP Trigger

Fires on an incoming HTTP request. Ideal for REST APIs and webhooks. Supports GET, POST, and custom routes.



Queue Trigger

Fires when a message arrives in Azure Queue Storage. Perfect for async background processing.



Blob Trigger

Fires when a blob is created or updated in Azure Blob Storage. Used for file processing pipelines.



Timer Trigger

Fires on a CRON schedule. Use for periodic tasks like cleanup jobs, reports, and health checks.

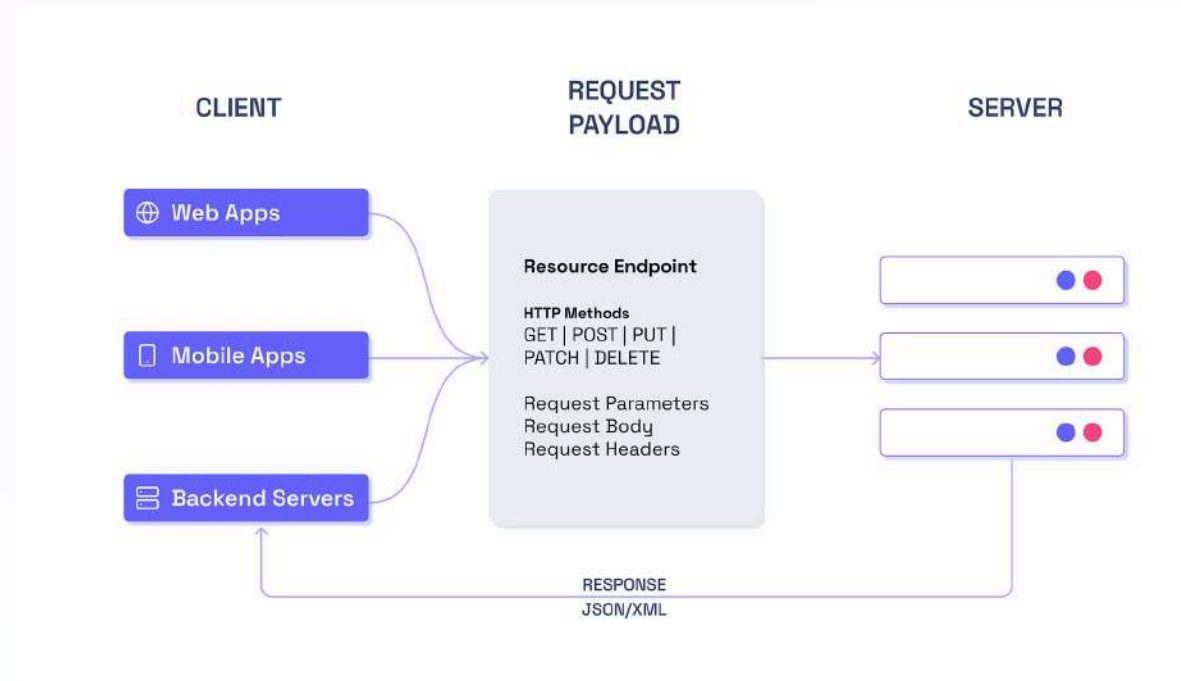
HTTP Trigger — REST APIs & Webhooks

How It Works

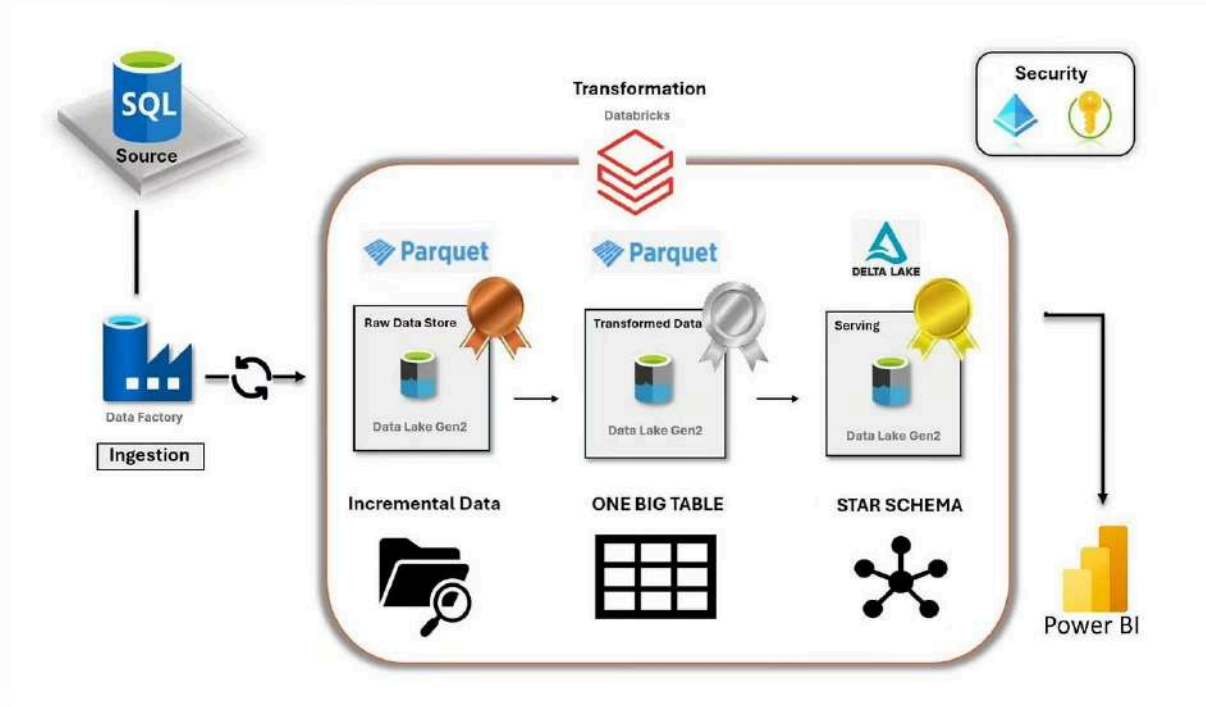
HTTP Triggers expose your function as an HTTP endpoint. You define the method (GET/POST), route template, and authorization level.

- **Anonymous** — no key required (dev/testing)
- **Function** — function-specific API key
- **Admin** — host-level master key

📄 HTTP Triggers are the fastest way to build lightweight serverless APIs without an API gateway.



Queue & Blob Triggers – Async Processing



Queue Trigger

- Polls Azure Queue Storage for new messages
- Automatically handles retries on failure (up to 5 by default)
- Poison message queue created after max retries exceeded

Blob Trigger

- Fires when blob is added or modified in a container
- Path pattern: `samples-workitems/{name}`
- Use Event Grid trigger for lower latency at scale

Timer Trigger — Scheduled Tasks

CRON Expression Format

Timer triggers use a 6-field NCRONTAB expression:

{second} {minute} {hour} {day} {month} {day-of-week}

0 0 * * * → every hour
0 */5 * * * → every 5 minutes
0 9 * * 1-5 → 9 AM weekdays

Timer triggers are ideal for **maintenance jobs**, **report generation**, **cache warming**, and **data sync tasks** that run on a predictable schedule.



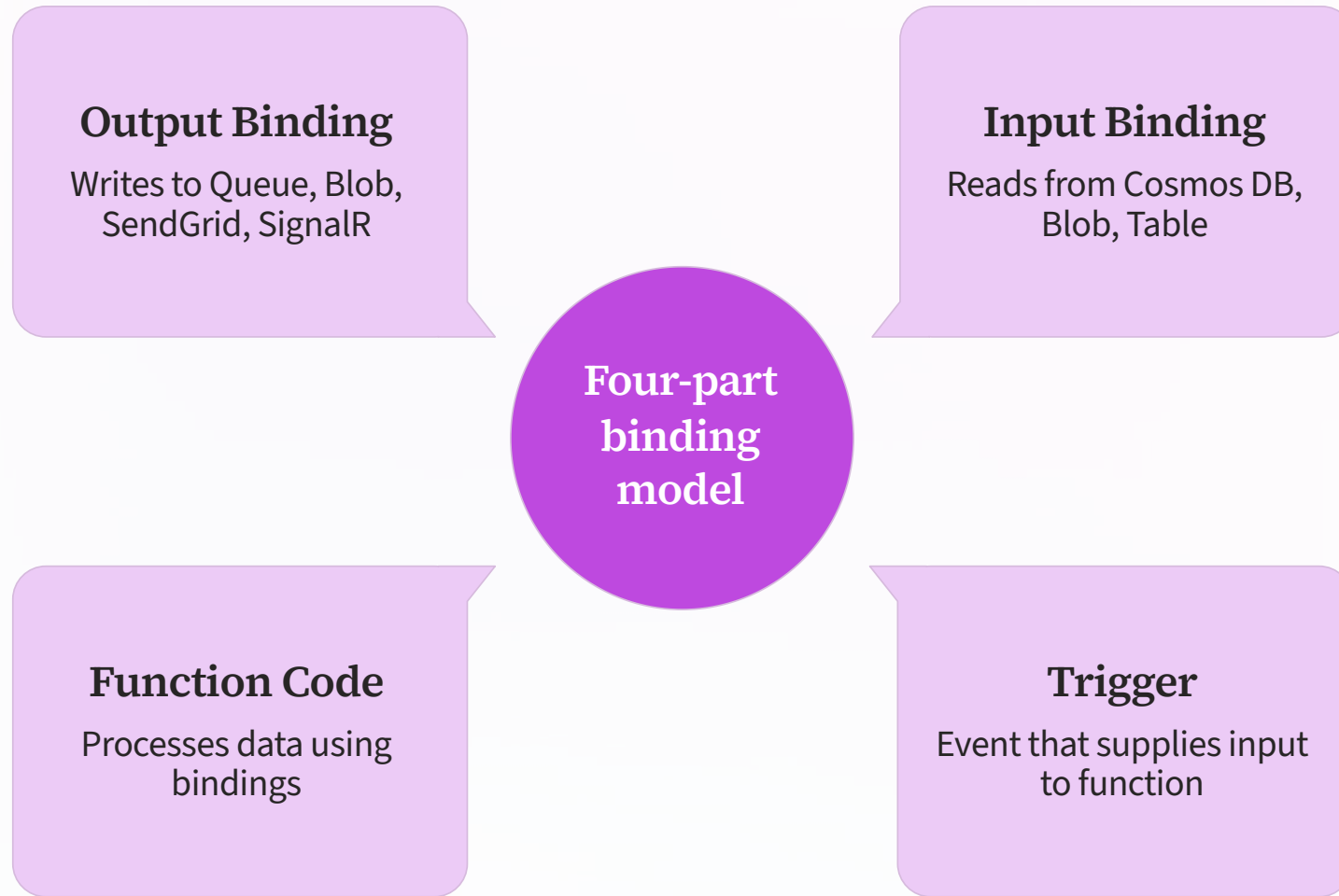


CHAPTER 5

Bindings

Input & Output Bindings

Bindings are **declarative connectors** to Azure services. Define them in configuration — no SDK boilerplate required in your function code.



Bindings dramatically reduce the code you write — connection strings, SDK clients, and retry logic are all handled by the runtime.

Common Binding Services



Cosmos DB

Read documents as input, write new documents as output.
Triggered by change feed too.



Blob Storage

Read a blob as input parameter. Write a new blob as output —
great for file transformation.



Queue / Service Bus

Output bindings enqueue messages. Decouple functions by
chaining them through queues.



SendGrid / Twilio

Send emails or SMS as output with a single binding attribute
— no client SDK needed.

CHAPTER 6

Hosting Plans

Choosing the Right Hosting Plan

Azure Functions offers three hosting plans. Your choice impacts **cost, cold start behavior, scale limits, and VNET access.**

1

Consumption

Fully serverless. Pay per execution.
Auto-scales to zero. Cold starts possible.
Best for sporadic workloads.

2

Premium

Pre-warmed instances. No cold starts.
VNET integration. Higher cost but
predictable performance.

3

Dedicated (App Service)

Runs on existing App Service Plan. No
auto-scale to zero. Best for always-on
workloads sharing infrastructure.

Hosting Plan Comparison

Feature	Consumption	Premium	Dedicated
Billing model	Per execution	Per instance/hour	App Service plan
Cold starts	Yes (possible)	No (pre-warmed)	No
Scale to zero	Yes	No	No
VNET integration	No	Yes	Yes
Max timeout	10 min	Unlimited	Unlimited
Best for	Sporadic events	Enterprise APIs	Shared infra

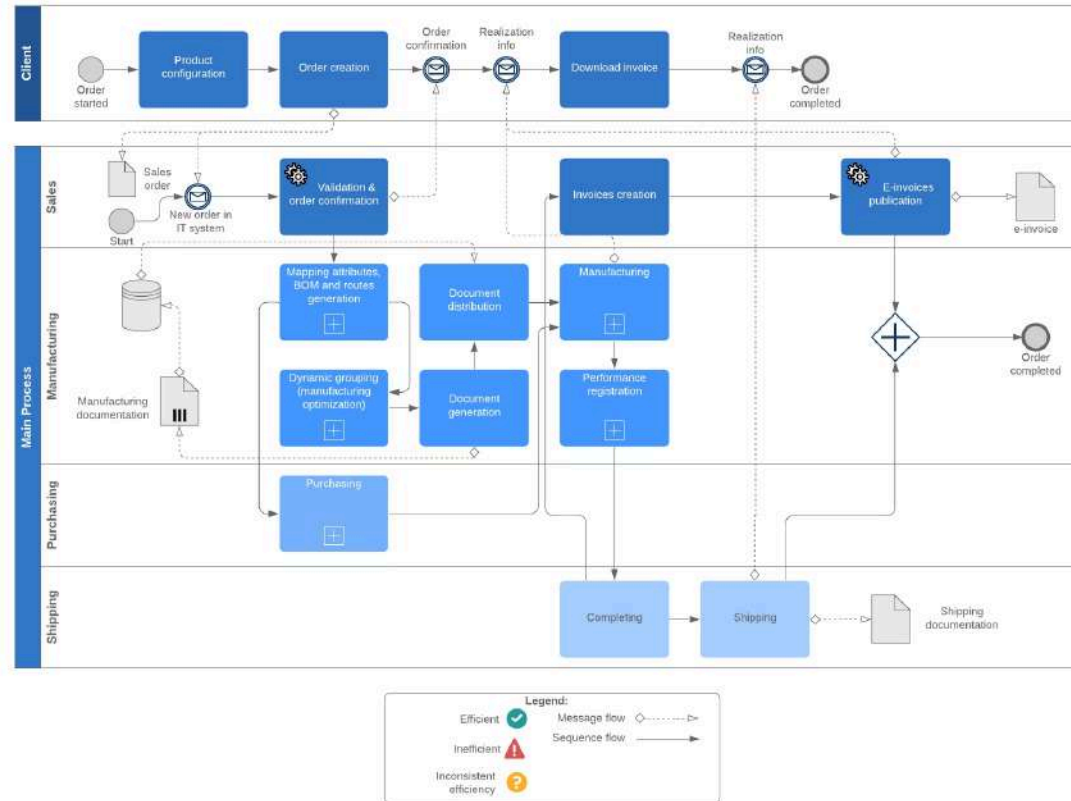
📌 The AZ-204 exam frequently tests when to choose Consumption vs. Premium — focus on VNET, cold start, and timeout constraints.



CHAPTER 7

Durable Functions

What Are Durable Functions?



Durable Functions is an extension of Azure Functions that lets you write **stateful workflows in code** — without managing state infrastructure yourself.

- State is automatically persisted to Azure Storage
- Functions can **wait for external events** or timers without consuming resources
- Supports fan-out/fan-in, human interaction, and chaining patterns
- Entire workflow defined in code — no visual designer required

❏ Durable Functions solve the hard problem: how do you coordinate long-running async work reliably?

Durable Functions: Key Components

1

Client Function

Starts (or queries) an orchestration instance. Triggered by HTTP, Queue, or any other trigger. Returns an instance ID.

2

Orchestrator Function

Defines the workflow logic. Calls activity functions, handles fan-out/fan-in, waits for timers or external events. **Must be deterministic.**

3

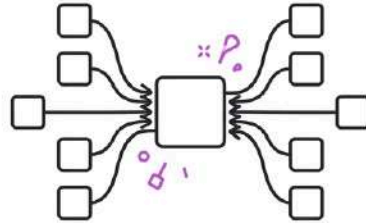
Activity Function

Does the actual work — calling APIs, writing to databases, processing data. Called by the orchestrator one or more times.

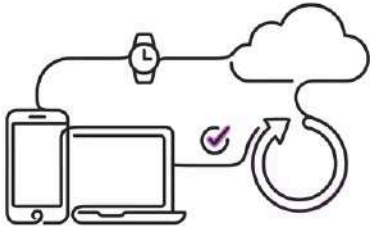
Durable Functions Patterns



**Function Chaining -
Sequential steps
A→B→C→D**



**Fan-Out/Fan-In -
Parallel tasks all
feeding to aggregator**



**Async HTTP API -
Client polls
orchestration status**



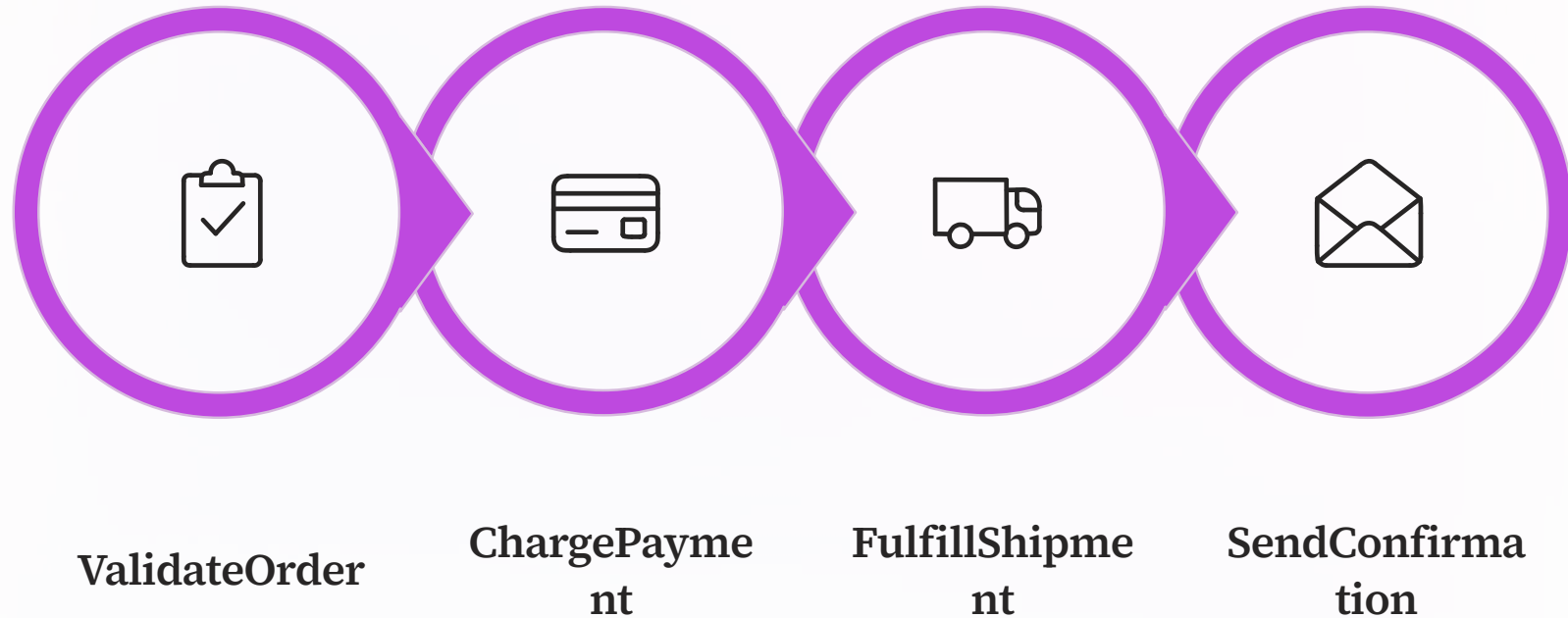
**Human Interaction -
Workflow waits for
approval event**

When to Use Each Pattern

- **Chaining** — sequential steps where each depends on the previous result
- **Fan-out/Fan-in** — process multiple items in parallel, then aggregate
- **Async HTTP API** — long-running job with status polling endpoint
- **Human Interaction** — approval workflows with configurable timeouts

❏ Durable Functions are the recommended approach for any workflow exceeding the 10-minute Consumption plan timeout.

Orchestration: Real-World Workflow Example



This four-step order workflow is coordinated by a single Durable Orchestrator function. If any step fails, Durable Functions retries automatically — and state is preserved across restarts.

Durable Functions: Orchestrator Rules

Orchestrator functions replay their execution history to rebuild state. This creates a **critical constraint: orchestrators must be deterministic.**

✓ Do This

- Call activity functions for all side effects
- Use `context.utcnowDateTime` for current time
- Use durable timers instead of `Thread.Sleep`



✗ Never Do This

- Call external APIs directly from the orchestrator
- Use `DateTime.Now` or `Guid.NewGuid()`
- Write to shared state or make non-deterministic calls

CHAPTER 8

End-to-End Scenarios



Scenario: File Processing Pipeline



This fully serverless pipeline requires **zero always-on infrastructure**. Each step scales independently — a spike in uploads scales the processing function automatically without impacting the notification step.

Scenario: Async Order Processing

The Pattern

Decoupling the order API from fulfillment logic using a queue makes the system resilient to downstream failures.

- HTTP Trigger receives order POST from frontend
- Output binding drops message on Azure Queue
- Queue Trigger Function processes asynchronously
- Durable Function orchestrates multi-step fulfillment

□ If the fulfillment service goes down, orders queue up safely — no data loss, automatic retry on recovery.

E-commerce Order Fulfillment Process



Scenario: Workflow Orchestration with Durable Functions

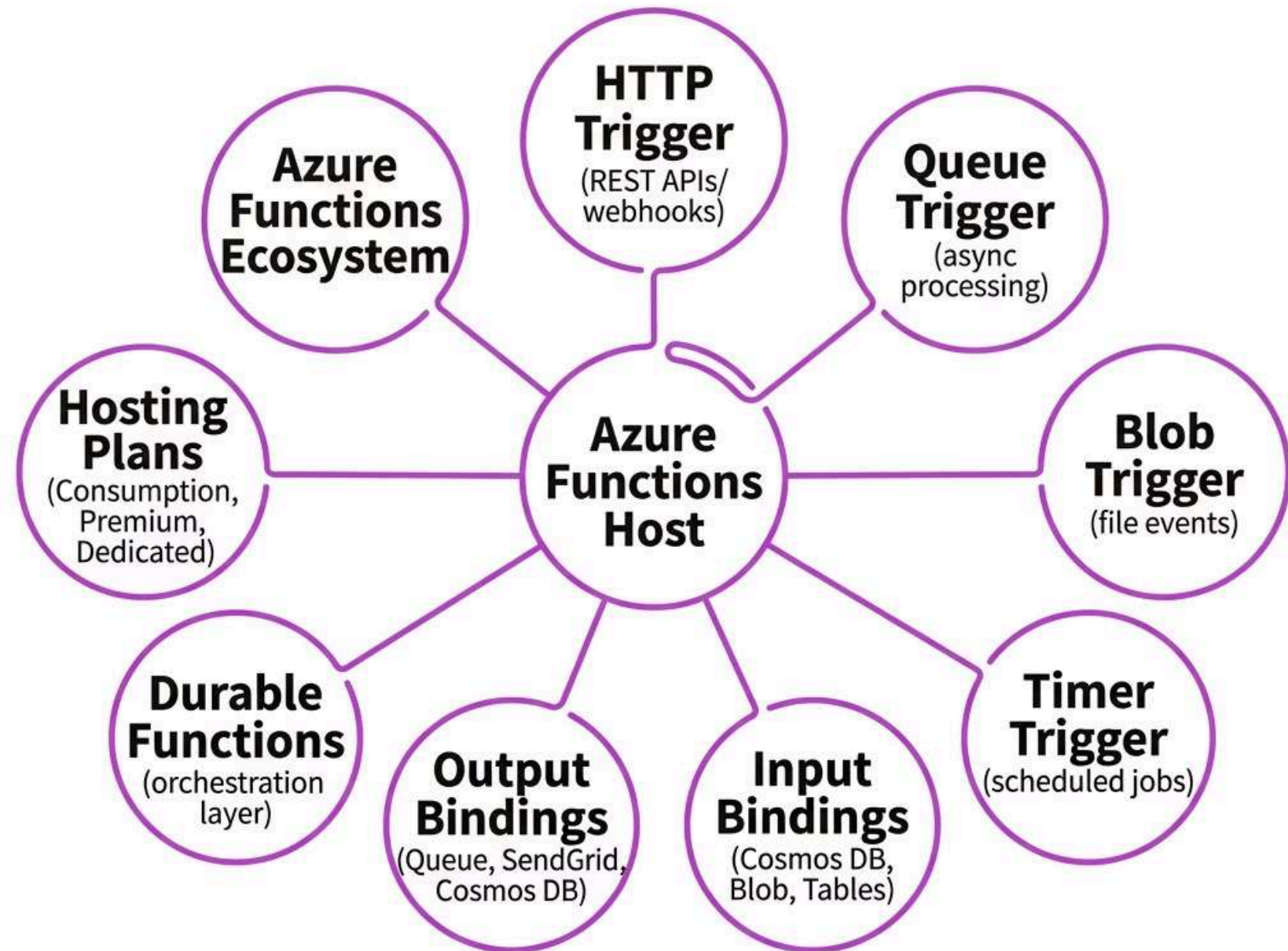


Human-in-the-Loop Approval

A multi-stage document approval workflow where a human must approve before the next step proceeds:

- Document submitted → orchestration instance started
- Orchestrator sends notification and **waits for external event**
- Reviewer approves or rejects via HTTP callback
- Durable timer fires escalation after 48 hours if no response
- Orchestrator proceeds or terminates based on decision

Architecture Summary: All Components Together



CHAPTER 9

Developer Best Practices



Azure Functions: Key Developer Principles

→ **Keep Functions Small and Focused**

One function, one responsibility. Large monolithic functions defeat the purpose of serverless decomposition.

→ **Use Managed Identity Over Connection Strings**

Avoid storing secrets in code or config. Use Managed Identity to authenticate to Azure services securely.

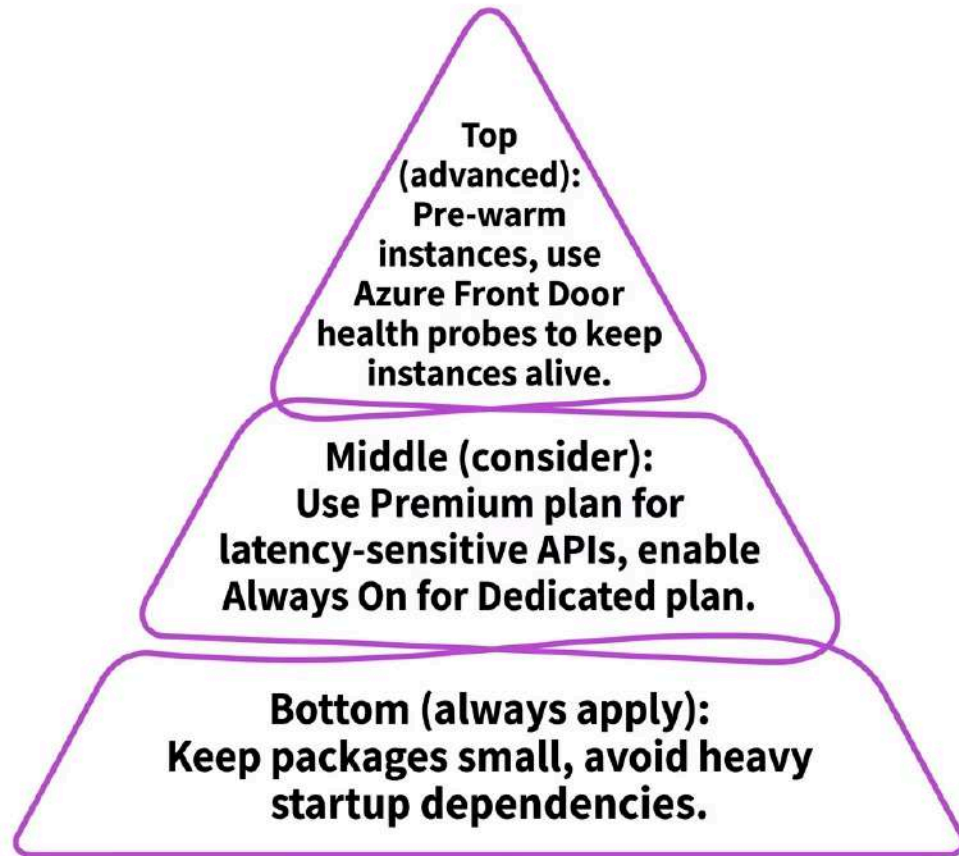
→ **Design for Idempotency**

Functions can be retried. Ensure the same input always produces the same result without side effects on re-execution.

→ **Monitor with Application Insights**

Enable Application Insights for every Function App. Distributed tracing is essential for diagnosing async workflows.

Cold Start Mitigation Strategies



Understanding Cold Starts

A cold start occurs when a new instance is created to handle a request after the function has scaled to zero.

- Typically adds **1–3 seconds** of latency
- .NET and Java have longer cold starts than Node.js/Python
- **Consumption plan** — cold starts unavoidable at zero scale
- **Premium plan** — pre-warmed instances eliminate cold starts

Key Metrics to Monitor

1M

Free Executions

Per month on Consumption plan — enough for most dev/test workloads at zero cost

10 min

Max Timeout

Consumption plan hard limit — use Durable Functions or Premium for longer workflows

200

Max Instances

Consumption plan scale-out limit per function app — Premium supports higher limits

5

Default Retries

Queue Trigger default retry count before a message is moved to the poison queue

CHAPTER 10

AZ-204 Exam Focus Areas



What the AZ-204 Exam Tests on Functions

High-Priority Topics

Trigger Selection

Know when to use HTTP vs. Queue vs. Blob vs. Timer based on a given scenario

Binding Configuration

Recognize correct binding attributes and direction (in/out/inout)

Hosting Plan Trade-offs

Consumption vs. Premium: cold starts, VNET, timeouts

Durable Patterns

Identify the correct pattern for chaining, fan-out, approval workflows

Common Exam Traps

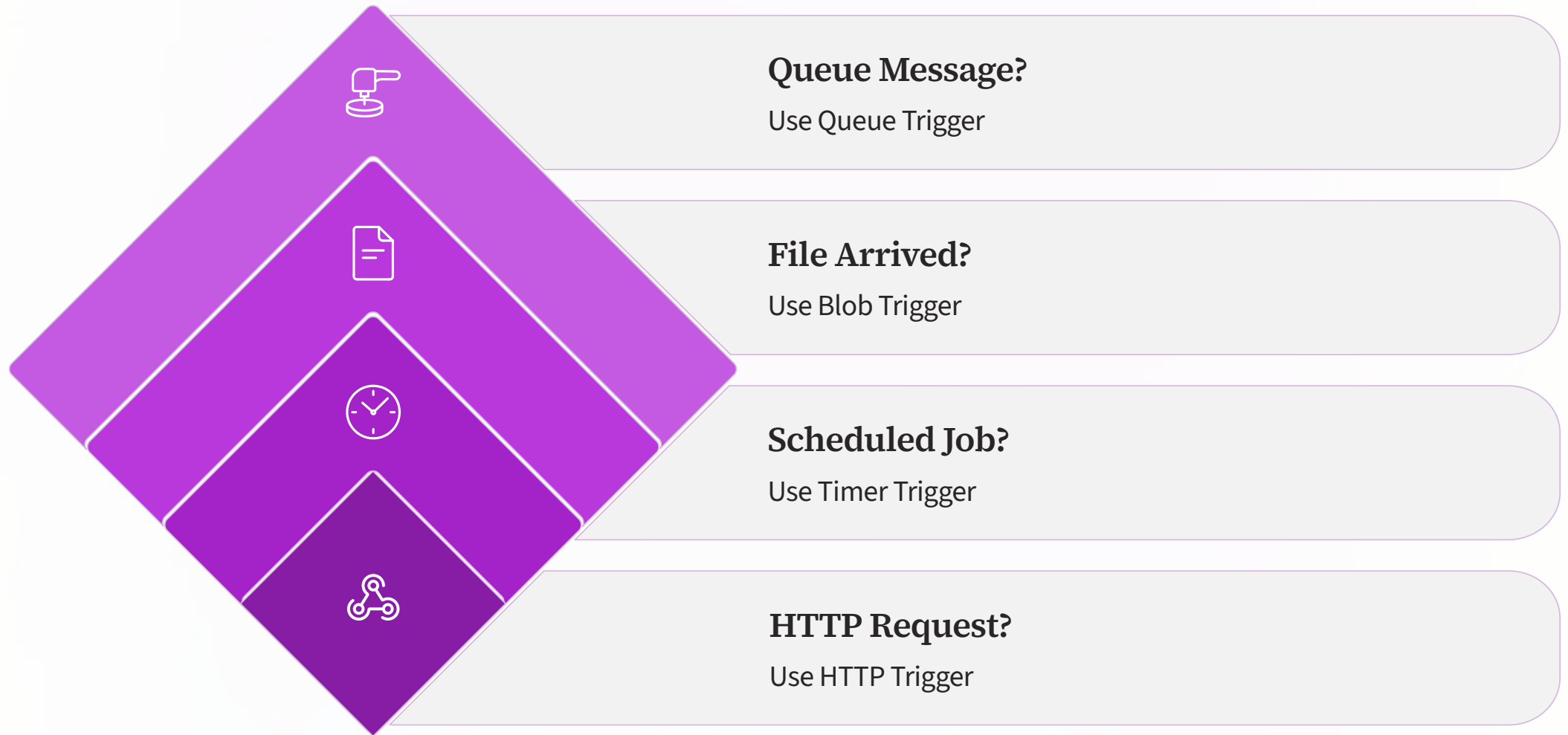
Blob Trigger has polling delay — use **Event Grid Trigger** for low-latency blob events at scale

Consumption plan timeout is **10 minutes max** — not configurable beyond that

Orchestrators must be **deterministic** — never call external APIs directly from them

A function can have **only one trigger** but **multiple input and output bindings**

Quick Reference: Trigger Decision Guide



Use this mental model during the exam: identify the **source of the event** first, then map it to the correct trigger type.

CHAPTER 11

Day 3 Labs

Day 3 Hands-On Labs

Four labs to reinforce today's concepts. Each lab builds on the previous — together they cover the full Azure Functions development lifecycle.

1

Lab 1

Create Azure Function App

2

Lab 2

Blob Trigger Function

3

Lab 3

Queue Trigger Function

4

Lab 4

Durable Functions Workflow

Day 3 Key Takeaways

Serverless = No Infra Management

Focus on business logic. Azure scales, retries, and manages the host runtime for you.

Triggers Drive Execution

Every function has exactly one trigger. Match the trigger type to the event source.

Bindings Eliminate Boilerplate

Declarative input/output bindings replace SDK clients and connection management code.

Durable Functions = Stateful Workflows

Use orchestrators for long-running, multi-step, and human-interaction workflows.

